

Esercizio 1

Scrivere un algoritmo **iterativo** che, dati in ingresso:

- due **pile** P1 e P2, contenenti lo stesso numero *n* di caratteri dell'alfabeto italiano, e
- un **carattere** CAR

inserisca in una terza pila P3 i caratteri presenti in P1 e in P2, prendendoli in modo **alternato** (un carattere da P1, uno da P2, uno da P1, ecc.). Il carattere considerato va inserito in P3 solo se è **diverso** da CAR.

Valutare poi i **tempi di calcolo** nei casi *migliore e peggiore*, indicando quante volte viene eseguita ogni operazione in pseudocodice. Si assuma che ogni singola operazione elementare impieghi lo stesso tempo (costante).

Esempio: date le pile (che vanno lette da sinistra verso destra, ad esempio G è l'elemento in cima alla pila P1):

P1 = A, B, A, A, C, A, D, D, A, F, A, G
P2 = R, I, B, A, M, A, L, D, A, B, A, R

e il carattere
CAR = A

il contenuto della pila P3, dopo aver eseguito l'algoritmo, sarà:
P3 = G, R, F, B, D, D, D, L, C, M, B, B, I, R

Spiegazione tempi di calcolo

procedure 9x1

P3 = nuova pila vuota
cor = "A"
counter = 0

while ! P1.Stack Empty or ! P2.Stack Empty

if counter % 2 == 0

if ! P1.Stack Empty

if P1.Top != cor

P3.push (P1.Pop)

else

P1.pop()

else

if ! P2.Stack Empty

if P2.Top != cor

P3.push (P2.pop())

else

P3.pop

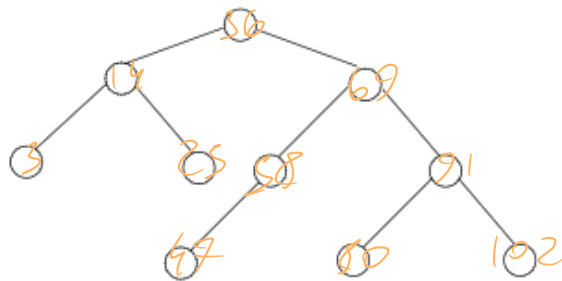
counter++ $\Theta(1)$ fatto 2 n volte

Esercizio 3

Data la sequenza:

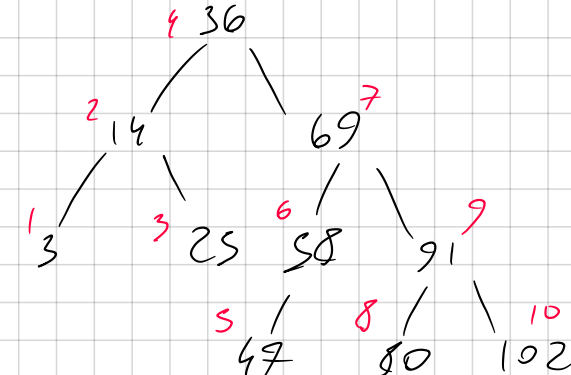
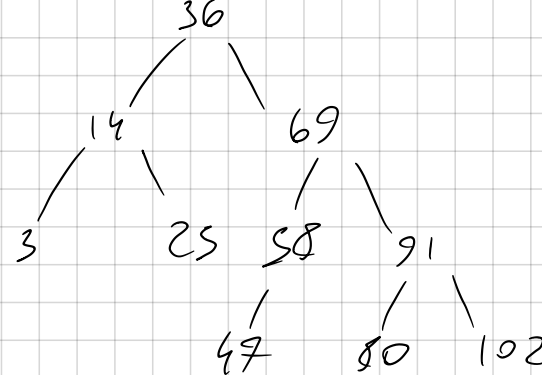
3, 14, 25, 36, 47, 58, 69, 80, 91, 102

di numeri interi, riempire la struttura ad albero data qui sotto in modo tale che risulti un albero binario di ricerca.

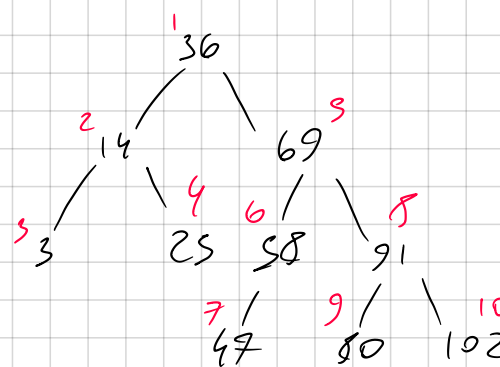


Si mostrino poi:

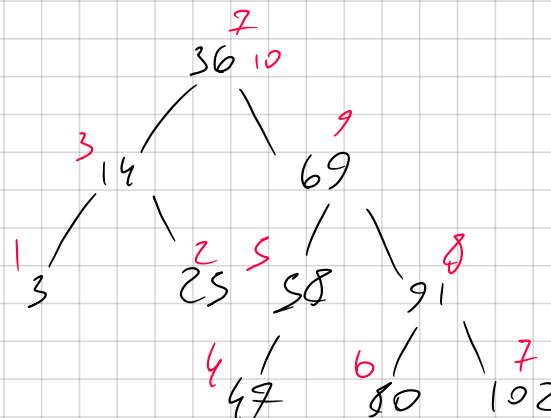
- le sequenze di interi ottenute visitando l'albero negli ordini **anticipato**, **simmetrico** e **posticipato** (ovvero, rispettivamente: pre-ordine, in-ordine e post-ordine);
- gli alberi ottenuti cancellando prima la chiave 36, poi la chiave 69 e infine la chiave 25 (mostrare la struttura dell'albero ottenuta dopo ogni singola cancellazione). Spiegare brevemente perché si ottengono tali alberi.



In-ordine visit: 3 14 25 36 47 58 69 80 91 102
(sx - centro - dx)

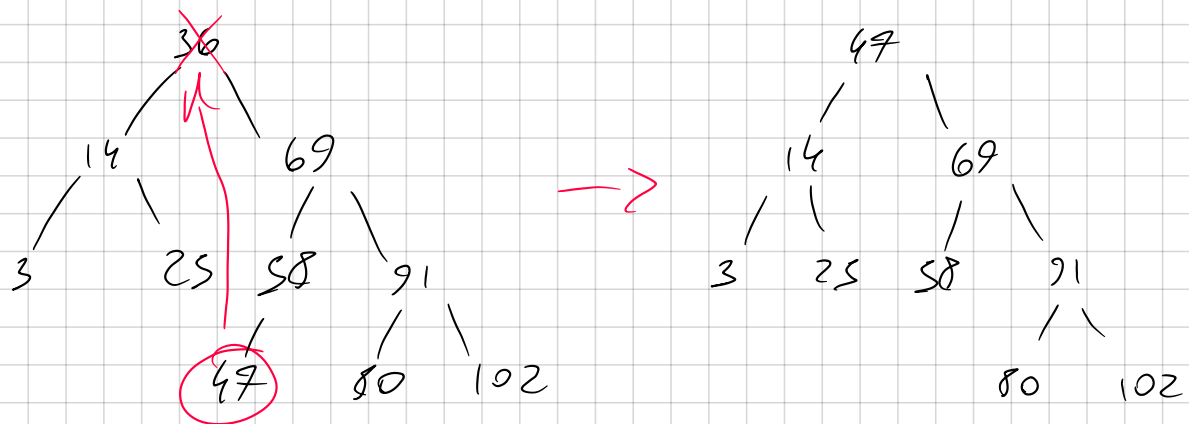


Pre-ordine visit: 36 14 3 25 69 58 47 91 80 102
(Centro - sx - dx)

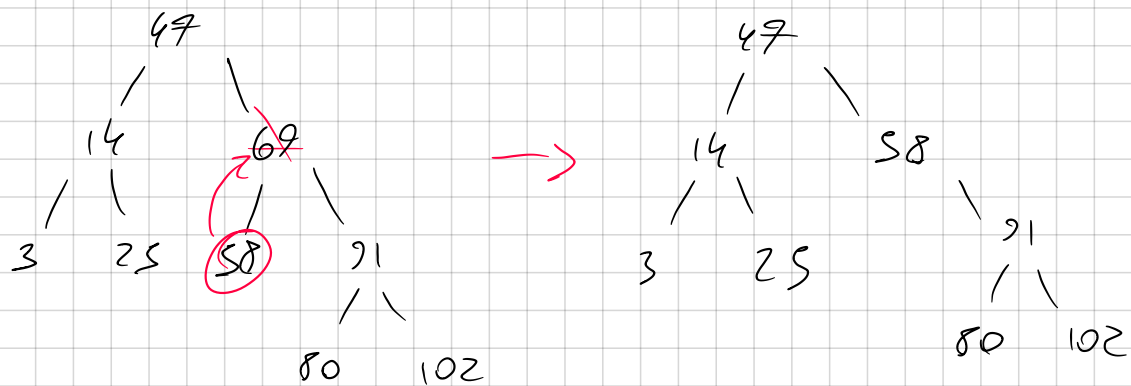


Post-ordine visit: 3 25 14 47 58 80 102 91 69 36
(sx - dx - Centro)

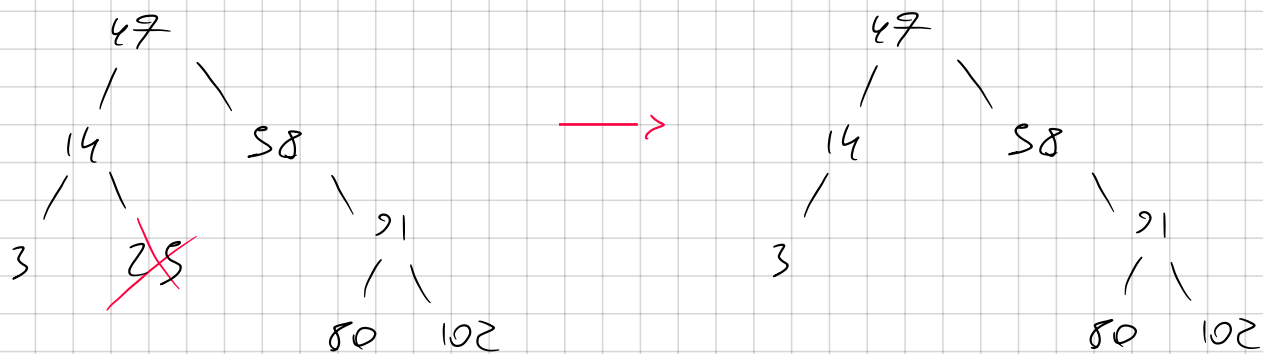
Rimuovi 36



Rimuovi 69



Rimuovi il 25



Esercizio 2

Simulare l'esecuzione della funzione BUILDHEAP sul seguente array:

A = [11, 41, 51, 67, 45, 46, 50, 63, 70, 60, 62, 74]

riportando lo stato dell'array **dopo ogni operazione di scambio** fra i suoi elementi, e illustrando i vari passaggi. L'array può essere rappresentato in forma di albero, se lo si ritiene opportuno, indicando in maniera chiara gli indici degli elementi.

	1	2	3	4	5	6	7	8	9	10	11	12
A =	11	41	51	67	45	46	50	63	70	60	62	74